REACT NATIVE NATIVE SHEET

Product concept, technical architecture, API design and delivery roadmap

Working package name

react-native-native-sheet

iOS	Android	React Native
Swift + UIKit	Kotlin + native Window/Dialog	TypeScript + native integration

A blueprint for a production-grade, native-presented bottom sheet framework. This document defines the intended product and architecture; it is not a claim that implementation or compatibility testing has already been completed.

Prepared: November 2026

1. Executive summary

Build a React Native library that exposes a consistent TypeScript API while delegating presentation, gestures, animation and lifecycle to native platform code. React Native remains responsible for rendering the sheet's application content; native code owns the presentation host and interaction mechanics.

The primary differentiator should be reliable multiple-sheet presentation: a second sheet can appear above the first without destroying the first sheet's React state. This is a technically demanding feature and should be treated as a core architecture requirement, not a cosmetic add-on.

Product goals

- Provide native-feeling sheet presentation on iOS and Android.
- Support stacked and nested sheets with predictable lifecycle and dismissal behaviour.
- Offer snap points, interactive gestures, scroll coordination and keyboard handling.
- Provide a clean TypeScript API with component and imperative usage patterns.
- Support React Native Legacy and New Architecture, with an Expo development-build workflow.
- Offer a browser fallback where practical, while clearly documenting platform differences.

Non-goals for the first release

- Perfect pixel-for-pixel parity between iOS and Android system sheets.
- A full navigation replacement or general-purpose window manager.
- Guaranteeing support for every third-party gesture, navigation or modal library without integration tests.
- Supporting arbitrary native content from unrelated view hierarchies inside a sheet.

2. Product positioning

Positioning: A native presentation framework for React Native, not merely another JavaScript bottom-sheet component.

Attribute	Product promise
Presentation	Native presentation host and platform-aware transitions.
Stacking	Multiple independently managed sheets with preserved underlying content.
Interaction	Native gesture handling, snap points and scroll coordination.
Developer experience	Typed API, predictable events, examples and clear setup instructions.
Compatibility	Legacy and Fabric integration, Expo development builds, and a web fallback.

3. Core use cases

- **Profile → Edit profile:** open an edit sheet over a profile sheet, save, dismiss the child and retain the parent state.
- **Checkout → Payment method → Confirmation:** present a sequence of sheets without losing checkout state.
- **Map discovery:** keep a map visible while a sheet expands and collapses.
- **Persistent mini sheet:** collapse media or task content into a compact docked panel, then expand it.
- **Form and keyboard:** keep focused inputs visible while the keyboard appears.

4. Feature specification

4.1 Presentation engine

- Present, dismiss, dismiss-all and replace operations.
- Stable sheet identifiers and a registry for named sheet definitions.
- Lifecycle events: will-present, did-present, will-dismiss, did-dismiss and on-state-change.
- Configurable backdrop, status-bar behaviour, corner radius and presentation style.
- Support interactive dismissal and programmatic dismissal.
- Defined behaviour when the presenting screen unmounts or the app backgrounds.

4.2 Snap points and sizing

- Named detents such as content, small, medium, large and full.
- Numeric point values and percentage-based heights.
- Initial snap point and controlled or uncontrolled snap selection.
- Dynamic content sizing with explicit minimum and maximum constraints.
- Recalculation after orientation, safe-area or content-size changes.
- Documented behaviour when content is taller than the available viewport.

4.3 Gesture and scrolling

- Drag to expand, collapse or dismiss.
- Velocity-aware settling and configurable thresholds.
- Coordinate sheet pan gestures with ScrollView, FlatList and nested scroll content.
- Prevent accidental dismissal while a child scroll view can still scroll upward.
- Handle gesture cancellation, interrupted animations and multitouch safely.
- Expose a way to disable pan-to-dismiss or lock the sheet at a detent.

4.4 Multi-sheet manager

Mode	Expected semantics
stack	Present a child sheet above the current sheet; preserve the parent and its React state.
replace	Replace the active sheet according to documented dismissal and cleanup rules.
overlay	Present a floating or overlay sheet without treating it as a child detent transition.
persistent	Keep a compact sheet mounted while allowing expansion and collapse.

Stacking requirements

- Independent IDs, content hosts, backdrop policy and dismissal state.
- Well-defined back-button and swipe-dismissal order (topmost sheet first by default).
- Restore focus to the parent after a child sheet is dismissed.
- Do not accidentally dismiss all sheets when only the topmost one should close.
- Define a maximum stack depth or configurable safeguard to prevent unbounded stacking.

- Ensure native view/controller/dialog cleanup when a sheet is removed.

4.5 Animation and appearance

- Native-driven transitions and interactive drag updates.
- Spring configuration, duration-based transitions and reduced-motion behaviour.
- Backdrop opacity, dimming, optional blur, corner radius, handle and shadow styling.
- Haptic feedback where supported and enabled.
- Avoid routing every gesture frame through the JavaScript thread.

4.6 Keyboard, navigation and accessibility

- Keyboard inset handling and focused-input visibility.
- Safe-area and edge-to-edge behaviour.
- Navigation transition and screen-unmount integration.
- VoiceOver and TalkBack labels, focus order and dismissal announcements.
- Dynamic type/font scaling, keyboard navigation and reduced motion.
- Status bar and system back behaviour documented per platform.

5. Proposed public API

Expose both an imperative manager API and a declarative component API. The imperative API should be the foundation for stacking and programmatic workflows; the component API should cover simple controlled visibility use cases.

5.1 Provider

```
import { NativeSheetProvider } from 'react-native-native-sheet';

export default function App() {
  return (
    <NativeSheetProvider>
      <RootNavigator />
    </NativeSheetProvider>
  );
}
```

5.2 Imperative API

```
import { useNativeSheet } from 'react-native-native-sheet';

function ProfileScreen() {
  const sheet = useNativeSheet();

  const openProfile = () => {
    sheet.present({
      id: 'profile',
      content: <ProfileContent />,
      snapPoints: ['50%', '90%'],
      initialSnapPoint: '50%',
    });
  };

  const openLocation = () => {
    sheet.present({
      id: 'location',
      content: <LocationPicker />,
      presentation: 'stack',
      snapPoints: ['70%', '95%'],
    });
  };

  return <Button title="Open profile" onPress={openProfile} />;
}
```

5.3 Manager operations

Operation	Purpose
present(config)	Present a new sheet using a unique ID and configuration.
dismiss(id?)	Dismiss a specified sheet or the topmost sheet when no ID is provided.
dismissAll()	Dismiss every active sheet in a deterministic order.
replace(id, config)	Replace a sheet while applying documented state and lifecycle rules.
snapTo(id, point)	Move a sheet to a named or numeric snap point.
expand(id)	Move a sheet to its largest configured detent.
collapse(id)	Move a sheet to its smallest configured detent.

5.4 Declarative component API

```
<NativeSheet
  visible={visible}
  snapPoints={['50%', '90%']}
  initialSnapPoint="50%"
  enablePanDownToClose
  onDismiss={() => setVisible(false)}
>
  <SheetContent>
    <ProfileForm />
  </SheetContent>
</NativeSheet>
```

5.5 Configuration types to define

- SheetConfig: id, content, presentation, snapPoints, initialSnapPoint, appearance and gesture settings.
- SheetAppearance: background, corner radius, backdrop, handle, shadow and blur configuration.
- SheetAnimation: animation type, damping, stiffness and duration.
- SheetEvents: presentation, dismissal, detent change, gesture state and errors.
- Platform-specific options should be isolated under an explicit platform configuration object.

API design rules

- Use stable string IDs; reject duplicate active IDs or define a clear update policy.
- Return a promise or operation result only when completion semantics are reliable and documented.
- Do not pass arbitrary React elements through a native bridge as serialized data; mount them in a React Native host view.
- Keep callbacks and state updates predictable when a sheet is dismissed by a native gesture.
- Avoid platform-specific names in the cross-platform core API unless behaviour genuinely differs.

6. Technical architecture

The central architecture boundary is between native presentation and React content rendering. Native code manages the presentation surface and interaction mechanics. React Native mounts and updates the sheet's content inside a native host container.

6.1 High-level flow

Layer	Responsibilities
Application	Screens, user state, business logic and sheet content.
TypeScript API	Provider, hooks, registry, validation, events and public types.
Native integration	Commands, events, native component host and architecture-specific glue.
iOS engine	UIKit presentation, view-controller lifecycle, gestures, detents and animations.
Android engine	Dialog/Window, custom container, touch handling, insets and animations.
Web fallback	DOM-based dialog/sheet with documented differences.

6.2 iOS design

Use Swift and UIKit. Evaluate UISheetPresentationController for system detents and standard system behaviour. For advanced stacking, custom detents, custom transitions or visual parity, use a custom presentation controller and animator where necessary.

```
ios/
  NativeSheetModule.swift
  NativeSheetManager.swift
  NativeSheetController.swift
  NativeSheetPresentationController.swift
  NativeSheetAnimator.swift
  NativeSheetGestureHandler.swift
  NativeSheetStack.swift
  NativeSheetHostView.swift
```

iOS engineering considerations

- Present from the correct active view controller and handle presentation-in-progress states.
- Define how a sheet host is attached to the React Native view hierarchy.
- Preserve React content identity when a child sheet is presented.
- Handle interactive transition cancellation and controller dismissal callbacks.
- Manage safe areas, keyboard notifications, trait changes and scene lifecycle.
- Avoid retaining view controllers or React views after dismissal.

6.3 Android design

Use Kotlin with a Dialog/Window-based presentation host and a custom sheet container for advanced behaviours. Android's standard BottomSheetDialog or Material components may be useful foundations, but should not be assumed to provide every required stacking or lifecycle behaviour.

```
android/src/main/java/.../nativesheet/
  NativeSheetModule.kt
  NativeSheetManager.kt
  NativeSheetDialog.kt
  NativeSheetContainer.kt
  NativeSheetBehavior.kt
  NativeSheetGestureHandler.kt
```

```
NativeSheetAnimator.kt
NativeSheetStack.kt
NativeSheetHostView.kt
```

Android engineering considerations

- Define window ownership, dim amount, focus, outside-touch and back-button behaviour.
- Coordinate edge-to-edge insets, navigation bars, status bar and IME/keyboard.
- Use a consistent gesture arbitration policy for nested scrolling.
- Ensure dialogs and windows are dismissed on Activity destruction or configuration change.
- Test multiple windows/dialogs across Android versions and OEM variations.

6.4 Legacy and Fabric

- Define the native view/component boundary before implementation.
- Use Codegen and Fabric native components for the New Architecture path.
- Provide the required Legacy integration path and avoid maintaining two unrelated product implementations.
- Keep the presentation manager's behaviour consistent while isolating architecture-specific registration and mounting.
- Test event ordering, commands and React host-view lifetime under both architectures.

7. Suggested repository structure

```
react-native-native-sheet/
■■■ src/
■   ■■■ components/
■   ■   ■■■ NativeSheet.tsx
■   ■   ■■■ SheetContent.tsx
■   ■   ■■■ SheetBackdrop.tsx
■   ■■■ hooks/
■   ■   ■■■ useNativeSheet.ts
■   ■   ■■■ useSheetState.ts
■   ■■■ manager/
■   ■   ■■■ SheetManager.ts
■   ■   ■■■ SheetRegistry.ts
■   ■■■ provider/
■   ■   ■■■ NativeSheetProvider.tsx
■   ■■■ types/
■   ■■■ fabric/
■   ■■■ web/
■   ■■■ index.ts
■■■ ios/
■■■ android/
■■■ plugin/
■   ■■■ withNativeSheet.ts
■■■ example/
■■■ docs/
■■■ tests/
■■■ package.json
■■■ react-native.config.js
■■■ app.plugin.js
■■■ tsconfig.json
```

The exact source layout should follow the React Native version and codegen conventions selected for the release. Avoid creating placeholder files solely to match this tree; each module should have a clear responsibility and tests.

8. Platform and ecosystem support

Environment	Target support	Notes
iOS	Native Swift	UIKit presentation host; test supported deployment targets.
Android	Native Kotlin	Dialog/Window or custom host; test API and OEM differences.
Legacy Architecture	Supported	Document minimum supported React Native versions.
New Architecture	Supported	Fabric component and TurboModule/Codegen integration as required.
Expo	Development builds	Config plugin and prebuild support; native code is not automatically available in Expo Go.
Web	Fallback	DOM implementation; document non-native behaviour and feature gaps.
TypeScript	First-class	Export public types and provide strict type checking.

Expo note

A third-party native library generally cannot run in the standard Expo Go binary unless that binary already contains the library. The supported path should be an Expo development build or a prebuilt native app with the config plugin applied.

9. Testing and quality strategy

Area	Representative tests
Presentation	Open, close, replace, interrupted transition, repeated presentation.
Stacking	Two, three and five sheets; dismiss top; restore parent; preserve state.
Gestures	Drag, velocity, snap selection, cancellation and disabled gestures.
Scrolling	ScrollView, FlatList, nested scroll and horizontal carousel interaction.
Keyboard	Focus, keyboard show/hide, resize/pan and input visibility.
Lifecycle	Navigation changes, app background, Activity/scene recreation and unmount.
Performance	JS thread pressure, repeated open/close, memory and dropped frames.
Accessibility	VoiceOver, TalkBack, focus restoration and reduced motion.
Compatibility	Legacy, Fabric, Expo development build, iOS and Android versions.
Web	Focus trap, Escape key, backdrop click, scroll lock and responsive sizing.

Quality gates before release

- No known crash or leaked native presentation host in repeated open/close tests.
- Topmost-only dismissal works consistently for stacked sheets.
- Parent React state survives child sheet presentation and dismissal.
- Gestures remain responsive when JavaScript is busy.
- No keyboard or safe-area regression in supported device configurations.
- Public API, limitations and platform differences are documented.
- CI builds and tests both supported React Native architectures.

10. Delivery roadmap

Phase	Scope	Exit criteria
0 — Design	API contract, supported RN versions, host-view strategy and stack semantics.	Architecture decision records and test plan approved.
1 — Core host	Provider, manager, one native sheet on iOS and Android.	Present/dismiss and content lifecycle work on both platforms.
2 — Interaction	Snap points, gestures, animations and scroll coordination.	Detents and nested scrolling pass integration tests.
3 — Stacking	Stack, replace, overlay and topmost dismissal.	Parent state preserved across nested sheet workflows.
4 — Production	Keyboard, safe areas, accessibility, lifecycle and error handling.	Quality gates pass on supported device matrix.
5 — Ecosystem	Fabric, Legacy, Expo plugin and web fallback.	Example apps and CI validate declared compatibility.
6 — Release	Docs, migration guide, changelog, npm packaging and support policy.	Versioned release candidate and published package.

11. Performance and reliability targets

- Keep drag and transition updates native-driven wherever possible.
- Avoid rerendering sheet content for every gesture frame.
- Aim for smooth 60 FPS on supported devices; 120 FPS is conditional on device, OS and rendering conditions.
- Release native controllers, dialogs, windows and host views deterministically.
- Keep the manager resilient to duplicate commands, stale IDs and asynchronous presentation races.
- Measure startup, present/dismiss latency, memory and frame pacing on real devices.

12. Risks and design decisions

Risk / question	Mitigation or decision required
React content mounting	Prove how React views are mounted into native presentation hosts on both architectures.
Multiple native presentations	Define stack ownership, parent-child lifecycle and platform-specific presentation rules.
Gesture conflicts	Create a documented gesture arbitration model and test common scroll components.
Navigation interaction	Test screen transitions, unmounts and presenting-controller validity.
Cross-platform parity	Specify shared semantics while allowing platform-native visual differences.
Expo expectations	Clearly distinguish Expo Go from development builds and config-plugin support.
Version matrix	Choose minimum RN, iOS and Android versions before promising compatibility.

13. Recommended MVP scope

The first public release should be intentionally smaller than the full vision. A stable, well-tested core is more valuable than a long list of partially working features.

- iOS and Android native presentation host.
- Present/dismiss API and controlled component API.
- Two or more snap points with drag interaction.
- One child sheet stacked above a parent with parent state preserved.
- Backdrop, corner radius, handle and basic spring animation.
- ScrollView and FlatList coordination.
- Keyboard and safe-area handling for common forms.
- Legacy and Fabric support for the explicitly declared RN version range.
- Expo development-build setup and a working example application.

Defer mini-player persistence, advanced blur/glass effects, complex map integration and elaborate web parity until the MVP's lifecycle and stacking behaviour are proven.

14. Release checklist

- Package name, ownership, license and contribution policy finalized.
- Peer dependencies and supported React Native versions declared.
- iOS podspec and Android Gradle integration validated in a clean app.
- Codegen and both architecture paths tested.
- Expo config plugin tested from a fresh prebuild.
- Example app demonstrates basic, nested, scrollable and keyboard sheets.
- README includes installation, API reference, limitations and troubleshooting.
- CI runs lint, typecheck, unit tests, native builds and integration tests.
- Semantic versioning, changelog and issue templates configured.
- Release candidate tested on physical iOS and Android devices.

15. Final recommendation

Build the library around a native presentation manager and a React content host. Make multiple-sheet stacking the signature capability, but prove the host-view lifecycle and parent-state preservation early. Keep the public API small, typed and consistent; add advanced presentation modes only after the core behaviour is reliable.

This blueprint is a product and engineering specification. It does not represent completed source code, verified performance numbers, or certified compatibility. Those claims should be made only after implementation and testing.